Finance Tracker

Prepared By : Pranav and Geetha

Enrollment No: EN12027098860

Enrollment No: EN12027097079

Batch code: DA804S80

INTRODUCTION

- A Finance Tracker is a Python application that helps users:
- Record income
- Record expenses
- Store data in CSV files
- Generate financial reports
- Track monthly spending

OBJECTIVES

- Record income and expenses.
- Store transactions in CSV files.
- Edit and delete transactions.
- Generate monthly reports.
- Generate category-wise reports.
- Calculate income, expenses, and balance.
- Apply Python programming concepts.

TECHNOLOGIES USED

- Python
- Pandas
- NumPy
- CSV Files
- File Handling
- Functions
- Dictionaries
- Classes(Budget class)
- Exception Handling
- Lists
- Loops
- Conditional Statements
- CSV module
- Text file
- Groq AI Integration

NumPy Statistics

NumPy is used to calculate:

- Mean Expense
- Mean Income
- Maximum Expense
- Minimum Expense
- Median Expense
- Standard Deviation of Expenses

Testing & Debugging

- Test Cases:
- Invalid Date
- Invalid transaction type
- Invalid category
- Invalid month format
- Invalid numeric input
- Empty Input
- Empty transaction list
- Non-existent transaction deletion
- Negative Amount
- Large Amount
- Invalid Menu Choice
- CSV File Missing

Functions Used

- Main menu
- welcome_menu()
- main_menu()
- transactions_menu()
- add_transaction()
- view_transactions()
- edit_transaction()
- delete_transaction()
- category_report()
- monthly_report()
- income_vs_expense_report()
- save_csv()
- load_csv()
- budgets_menu()
- reports_menu()
- search_transaction() (if present)
- set_budget()
- get_budget()
- remaining_budget()
- numpy_statistics()
- AI advisor function()

Main Module (main.py)

- Acts as the entry point of the Personal Finance Tracker.
- Imports all project modules:
- Budget
- Transactions
- Reports
- Data Manager
- Loads previously saved transaction and budget data.
- Displays the Welcome Menu and Main Menu.
- Controls navigation between Transactions, Budgets, and Reports modules.
- Validates user input using try-except.
- Uses while loops for continuous menu execution.
- Uses if-elif-else for menu selection.
- Creates the Budget object using OOP concepts.

```python
# = are total 7, in each side of every menu

from budget import Budget
import transactions as t   # Importing all functions related to transactions
                           # from the transactions.py file
import reports as r
import data_manager as dm
import ai_advisor as ai
import analysis


'''
Pseudo test purposes list and dict format

transactions = [
    { eg.
        "date": "17-07-2026",
        "type": "Expense",
        "category": "Food",
        "amount": 250,
        "description": "Lunch"
    }
]
'''

transactions = dm.load_transactions ()

# print("Transactions loaded:", len(transactions))

budget = Budget ()

saved_budget = dm.load_budget ()

budget.set_budget ( saved_budget )



def welcome_menu() :

    while True :

        print("\n\n======= PERSONAL FINANCE TRACKER =======")
        # print("1. Login")
        # print("2. Register")

                # Version 2 Idea:
                # Add user authentication with multiple accounts,
                # usernames, and passwords.

        print("1. Start.\n")
        print("2. Exit\n") # originally 3.

        try:
            return int(input("Enter your choice: "))

        except ValueError:
            print("Please enter a number.")


def main_menu() :

    while True :

        print("\n\n======= MAIN MENU =======")
        print()

        print("1. Transactions.")
        print("2. Budgets.")
        print("3. Reports.")
        print("4. Analysis.")
        print("5. AI Advisor.")
        # print("6. Logout.") for ver 2
        print("6. Back to Home.")
        print()

        try :
            main_choice = int(input("Enter choice: "))
        except ValueError :
            print("Please choose a number!")
            continue


        if main_choice == 1:
            transactions_menu()

        elif main_choice == 2:
            budgets_menu()

        elif main_choice == 3:
            reports_menu()

        elif main_choice == 4:
            analysis.financial_analysis ()

            input("\nPress Enter to return...")

        elif main_choice == 5:
            ai.ai_advisor ( transactions, budget )

        elif main_choice == 6:

            sure = input("\nAre you sure you wish to go back to home page? ( Y / N ): ").lower()
            # back to home page == logout

            if sure == 'y' :
                print("\nYou have successfully Logged Out from your Personal Finance Tracker!\n")
                break

            elif sure == 'n' :
                continue

            else :
                print("Please enter either Y or N !")

```

Transactions

- Manages all financial transactions (Income & Expense).
- Uses **functions** for Add, View, Edit, and Delete operations.
- Stores data using a **list of dictionaries**.
- Validates user input using **try-except**.
- Uses **while loops** for continuous menu execution.
- Uses **if-elif-else** for menu selection and validation.
- Saves updated transactions to **CSV** using Pandas.

TRANSACTIONS-OUTPUT

```
In [1]: %runfile C:/Users/Murugappan/Downloads/main.py --wdir


======= PERSONAL FINANCE TRACKER =======
1. Start.

2. Exit

Enter your choice: 1


======= MAIN MENU =======

1. Transactions.
2. Budgets.
3. Reports.
4. Back to Home.
```

```
Enter choice: 1


======= ADD TRANSACTIONS =======

Enter Date: 08-08-2026

Enter Type (Income/Expense): income
Enter Category: bank interest

Enter Amount: 500
Enter Description: internet bill

Transaction added successfully!
```

```
======= EDIT TRANSACTIONS =======

1. Date.
2. Type.
3. Category.
4. Amount.
5. Description.
6. Cancel.

Enter choice: 3
Enter New Category: Shopping
Transaction updated successfully!
```

```
Enter choice: 1


======= TRANSACTIONS MENU =======

1. Add Transaction.
2. View Transactions.
3. Edit Transaction.
4. Delete Transaction.
5. Back.

Enter choice: 1


======= ADD TRANSACTIONS =======

Enter Date: 08-08-2026

Enter Type (Income/Expense): income
Enter Category: bank interest
```

```
Enter choice: 2

Transaction 1:
---------------------------------------
Date        : 01-07-2026
Type        : Income
Category    : Salary
Amount      : 50000
Description : Monthly Salary
---------------------------------------

Transaction 2:
---------------------------------------
Date        : 02-07-2026
Type        : Expense
Category    : Food
Amount      : 250
Description : Lunch at Domino's
---------------------------------------
```

```
---------------------------------------
Date        : 08-08-2026
Type        : Income
Category    : Shopping
Amount      : 500
Description : internet bill
---------------------------------------

Enter transaction number: 50
Are you sure you want to delete this transaction? (Y/N): y

Transaction deleted successfully!
```

BUDGET

Budget Module

- Implements **Object-Oriented Programming (OOP)**.
- Uses a **Budget class** to manage monthly budgets.
- Uses **Constructor (__init__)** for initialization.
- Uses **Setter** method to set the budget.
- Uses **Getter** method to retrieve the budget.
- Calculates remaining budget after expenses.
- Displays budget status (Within Budget / Exceeded Budget).



```
class Budget :

    def __init__(self) :
        """ Constructor """
        self.monthly_budget = 0

    def set_budget ( self, amount ) :
        """ Setter """
        self.monthly_budget = amount

    def get_budget ( self ) :
        """ Getter """
        return self.monthly_budget

    def budget_status ( self, transactions ) :

        """ Sets budget status using transactions list """

        if self.monthly_budget == 0 :
            print("No monthly budget has been set yet!")
            return

        total_expenses = 0

        for transaction in transactions :

            transaction_type = transaction["Type"]
            amount = transaction["Amount"]

            if transaction_type == "Expense" :
                total_expenses += amount

        remaining_budget = self.monthly_budget - total_expenses

        print("\n\n======= BUDGET STATUS =======")
        print()

        print(f"\nMonthly Budget : Rs. {self.monthly_budget:.2f}")
        print()

        print(f"Total Expenses :  Rs. {total_expenses:.2f}")

        print()

        if remaining_budget >= 0 :
            print(f"Remaining Budget : Rs. {remaining_budget:.2f}")
            print("Status : Within Budget!\n")

        else :
            remaining_budget = abs( remaining_budget )
```

BUDGET OUTPUT

```
Enter choice: 2


======= BUDGETS MENU =======

1. Set Budget.
2. View Budget.
3. Budget Status.
4. Back.

Enter a choice: 1

Enter the Monthly Budget: Rs. 100

Monthly Budget Set to RS. 100.00
```

```
Enter the Monthly Budget: Rs. 200

Monthly Budget Set to RS. 200.00
```

```
Enter a choice: 3


======= BUDGET STATUS =======


Monthly Budget : Rs. 200.00

Total Expenses :  Rs. 65507.00

Exceeded Budget by : Rs. 65307.00
Status : Exceeded Budget!
```

REPORT

- Generates financial reports from stored transactions.
- Monthly Report filters transactions by month.
- Category Report groups expenses by category.
- Income vs Expense Report calculates total income, expenses, and balance.
- Uses loops and dictionaries for report generation.
- Displays summarized financial information.

```
# ANYTHING RELATED TO REPORTS MUST BE PUT HERE


def monthly_report ( transactions ) :

    if not transactions :
        print("No Transactions Found!")
        return

    print("\n======= MONTHLY REPORT =======")
    print()

    month = input("Enter Month and Year (MM-YYYY): ")

    found = False

    for transaction in transactions :
        date = transaction["Date"]

        transaction_month = date.split("-")[1] + "-" + date.split("-")[2]

        if month == transaction_month :
            found = True

            for key, value in transaction.items() :
                print(f"{key:<15}: {value}")

            print("----------------------------------------")
            print()

    if not found :
        print("No transactions found for this month.\n")


def category_report ( transactions ) :

    if not transactions :
        print("No Transactions Found!")
        return

    print("\n======= CATEGORY REPORT =======")
    print()


    category_total = {}

    for transaction in transactions :

        category = transaction["Category"]
        amount = transaction["Amount"]

        if category in category_total :
            category_total[ category ] += amount

        else :
            category_total[ category ] = amount

    for category, amount in category_total.items() :
        print(f"{category:<15}: Rs. {amount:.2f}")
```

```
def income_vs_expense_report ( transactions ) :

    if not transactions :
        print("No Transactions Found!")
        return

    print("\n======= INCOME V/S EXPENSE REPORT =======")
    print()

    total_income = 0
    total_expense = 0

    for transaction in transactions :

        if transaction["Type"] == "Income" :
            total_income += transaction["Amount"]


        elif transaction["Type"] == "Expense" :
            total_expense += transaction["Amount"]


    balance = total_income - total_expense

    print(f"Total Income   : Rs. {total_income:.2f}\n")
    print()

    print(f"Total Expense  : Rs. {total_expense:.2f}\n")
    print()

    print(f"Net Balance    : Rs. {balance:.2f}\n")
    print()
```

REPORT -OUTPUT

```
======= REPORTS MENU =======

1. Monthly Report.
2. Category Report.
3. Income v/s Expense.
4. Back.

Enter a choice: 1

======= MONTHLY REPORT =======

Enter Month and Year (MM-YYYY): 07-2026
Date            : 01-07-2026
Type            : Income
Category        : Salary
Amount          : 50000
Description     : Monthly Salary
-------------------------------------

Date            : 02-07-2026
Type            : Expense
```

```
Enter a choice: 2

======= CATEGORY REPORT =======

Salary          : Rs. 200000.00
Food            : Rs. 2310.00
Travel          : Rs. 1400.00
Internet        : Rs. 2697.00
Freelance       : Rs. 23000.00
Shopping        : Rs. 10200.00
Electricity     : Rs. 4550.00
Interest        : Rs. 1300.00
Medical         : Rs. 3250.00
Gift            : Rs. 7000.00
Rent            : Rs. 36500.00
Entertainment   : Rs. 3550.00
Scholarship     : Rs. 16000.00
Miscellaneous   : Rs. 1550.00
```

```
Enter a choice: 3

======= INCOME V/S EXPENSE REPORT =======

Total Income    : Rs. 247800.00


Total Expense   : Rs. 65507.00


Net Balance     : Rs. 182293.00
```

NumPy

```python
import pandas as pd
import numpy as np

# Read CSV file
df = pd.read_csv("transactions.csv")

# Convert Amount column to numeric
df["Amount"] = pd.to_numeric(df["Amount"])

# Convert Date column to datetime
df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y")

# ------------------------------
# Total Income
# ------------------------------
total_income = df[df["Type"] == "Income"]["Amount"].sum()
print("Total Income :", total_income)

# ------------------------------
# Total Expenses
# ------------------------------
total_expense = df[df["Type"] == "Expense"]["Amount"].sum()
print("Total Expenses :", total_expense)

# ------------------------------
# Current Balance
# ------------------------------
balance = total_income - total_expense
print("Current Balance :", balance)

# ------------------------------
# Expenses by Category
# ------------------------------
print("\nExpenses by Category")
expense_category = df[df["Type"] == "Expense"].groupby("Category")["Amount"].sum()
print(expense_category)

# ------------------------------
# Income by Category
# ------------------------------
print("\nIncome by Category")
income_category = df[df["Type"] == "Income"].groupby("Category")["Amount"].sum()
print(income_category)

# ------------------------------
# Monthly Summary
# ------------------------------
df["Month"] = df["Date"].dt.strftime("%B")
```

```
utils.py (209): exec_fun(compile(code_ast, filename, "exec"), globals, None)
untitled1.py (3): from budget import Budget — Scope: <module>
```

Help | Variable Explorer | Debugger | Profiler | Plots | Files

Console 1/A ×

```
In [1]: %runfile C:/Users/Murugappan/Downloads/numpy.py --wdir
Total Income : 247800
Total Expenses : 65507
Current Balance : 182293

Expenses by Category
Category
Electricity       4550
Entertainment     3550
Food              2310
Internet          2697
Medical           3250
Miscellaneous     1550
Rent             36000
Shopping         10200
Travel            1400
Name: Amount, dtype: int64

Income by Category
Category
```

AI Financial Advisor

- Uses Groq API
- Generates spending insights
- Gives budgeting suggestions
- Suggests saving opportunities

```python
# AI ADVISOR OF YOUR FINANCE TRACKER

from groq import Groq

# adding AI client - GROQ
client = Groq ( api_key = "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")


def ai_advisor ( transactions, budget ) :

    if not transactions:

        print("No transactions found!")
        return

    if budget.get_budget() == 0 :

        print("No monthly budget has been set yet!\n")
        return


    print("\n======= FINANCIAL AI ADVISOR =======\n")
    print()

    print("Generating personalized financial advice...\n")
    print("Please wait...\n")

    # print(f"Loaded Transactions : {len(transactions)}")
    # print(f"Current Budget      : Rs. {budget.get_budget():.2f}")

    # to get faster responses from AI its better to give a concise summary to
    # the AI. It will ensure the user always gets good quality responses from the user

    total_income = 0
    total_expense = 0

    expense_categories = {}

    for transaction in transactions :

        if transaction["Type"] == "Income" : # Calculating Total Income

            total_income += transaction["Amount"]

        elif transaction["Type"] == "Expense" : # Calculating Total Expense

             total_expense += transaction["Amount"]
```

Conclusion

- Statistical Analysis of Profit & Loss
- **Key Statistics**
- Total Income: **₹247,800**
- Total Expenses: **₹65,507**
- Net Profit: **₹182,293**
- Profit Margin: **73.56%**
- Expense Ratio: **26.44%**

- **Interpretation**
- The Finance Tracker generated a **net profit of ₹182,293**.
- Expenses account for **26.44%** of total income.
- The remaining **73.56%** represents the available profit, indicating healthy financial management.

THANK YOU